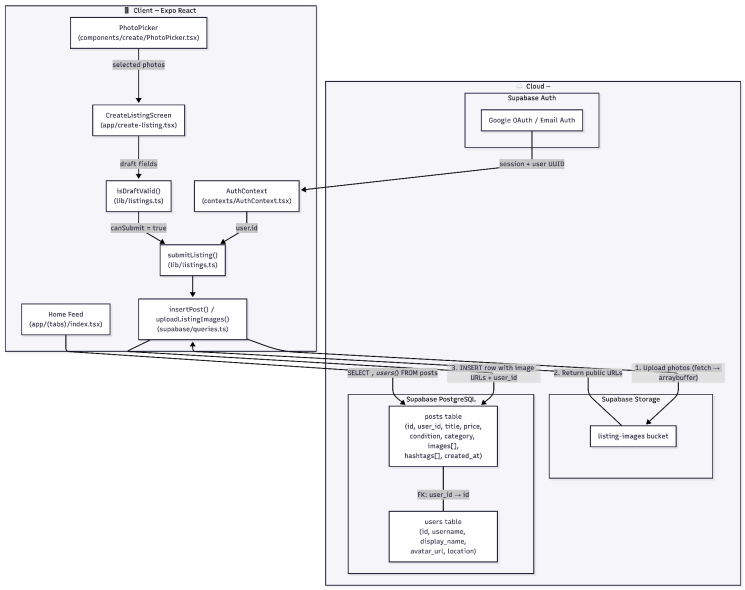
User Story 1: Quick Listing Creation

User Story:
As a seller, I want to quickly snap photos and list an item with minimal text so that I can post my second-hand goods for sale without it feeling like a chore.

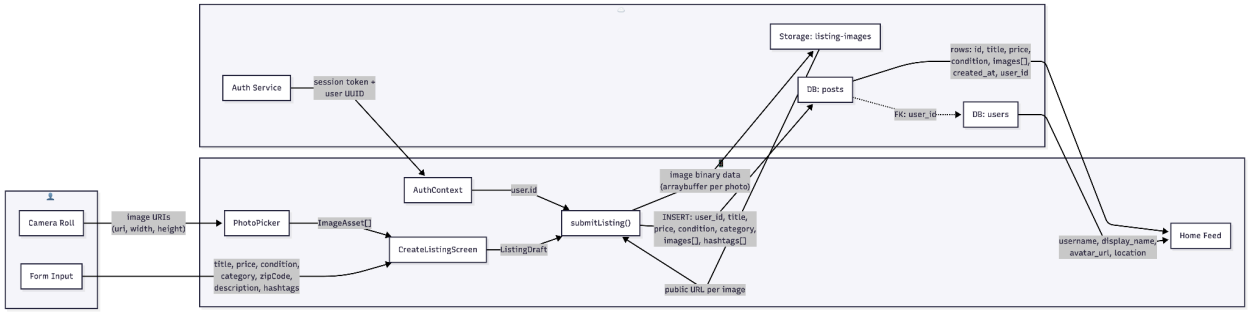
Primary Owner: Noah
Secondary: Zach

Date merged: March 24th, 2026

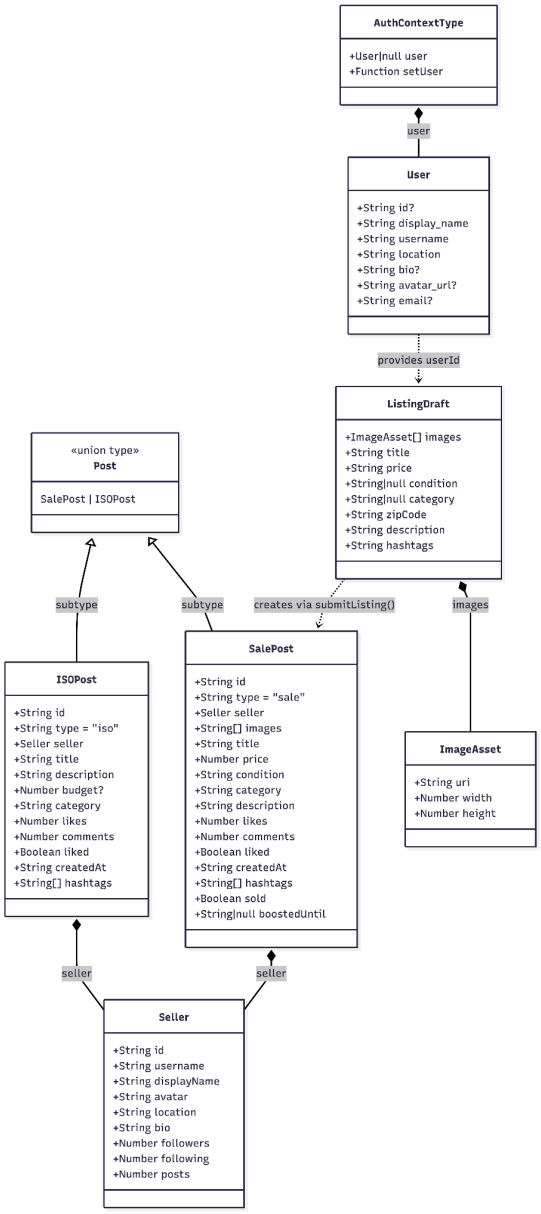
An architecture diagram in Mermaid that also illustrates where the components of the application execute (e.g., client, server, cloud, etc.).



An information flow diagram in Mermaid that shows which user information and application data moves between architectural components and the direction in which they flow.



A class diagram in Mermaid that shows the classes relevant to the user story's implementation in superclass/subclass relationships.



A list of all of the classes in the implementation relevant to this user story. Each class must include a list of public and private fields and methods with explanations for the purpose for each. First list the public fields and methods together, then list the private fields and methods. Each list should be grouped by concept.

For the quick-listing story—snap photos, enter minimal text, and publish a sale without it feeling like a chore—the pieces that matter are these.

ImageAsset is the shape of each picked photo: uri is where the app reads bytes for upload; width and height come from the picker as metadata. There are no hidden members on that interface.

ListingDraft is the in-memory form: images holds the ImageAsset[] before anything hits the server; title and price (as a string for the input) are the core text fields; condition and category are either a chosen value or null until the user taps a chip; zipCode is five digits and is validated in the app; description and hashtags stay optional so the flow stays light.

The listing module in lib/listings.ts exposes EMPTY_DRAFT as the initial state, CONDITIONS and CATEGORIES for the chip sets, and MAX_PHOTOS as the cap shared with the picker. isDraftValid turns on submit only when there is at least one photo, a non-empty title, a valid price, both condition and category chosen, and a five-digit zip. submitListing forwards the draft and optional userId into insertPost. parseHashtags is there if you normalize tag text on the client; the server path still normalizes hashtags when inserting.

PhotoPicker is a controlled child: the parent passes images and an onChange that updates draft.images. Inside, it uses theme colors, computes how many slots are left under MAX_PHOTOS, and implements pickFromGallery (multi-select to ImageAsset objects), takePhoto (camera permission then one capture), and removePhoto (filter out an index)—all ending in onChange so the draft stays the single source of truth.

User and auth exist so the listing is attributed to a real seller: id is what becomes user_id on the row when you call submitListing(draft, user?.id). AuthProvider holds that state; useAuth is how the create screen reads user. Nothing else on the auth type is required for the story beyond having an id when available.

CreateListingScreen is the default export that wires everything together: it keeps draft and submitting in React state, uses refs on the text fields to chain focus, derives canSubmit from isDraftValid, patches the draft with a small update helper, and on success runs handleSubmit which calls submitListing, then shows feedback and goes back. Theme, safe area, and router are only there to present the form cleanly.

A complete list of technologies/libraries/APIs used in your system that you aren't writing yourself.

Language & tooling

TypeScript ~5.9.2 — Static typing for the whole app and safer refactors. Why: Catches errors at compile time and improves editor support vs plain JavaScript. Source: https://github.com/microsoft/TypeScript — Author: Microsoft — Docs: https://www.typescriptlang.org/docs/

Node.js (version not pinned in this repo; follow Expo SDK 54 requirements, typically Node 20+). — Runs npm, Metro tooling, and build scripts. Why: Standard JS runtime for React Native / Expo development. Source: https://github.com/nodejs/node — Author: OpenJS Foundation — Docs: https://nodejs.org/docs/

npm (via package-lock.json) — Installs dependencies and locks versions. Why: Default with Node; reproducible installs vs unpinned installs. Source: https://github.com/npm/cli — Author: npm / GitHub — Docs: https://docs.npmjs.com/

ESLint ^9.25.0 + eslint-config-expo ~10.0.0 — Linting aligned with Expo. Why: Consistent style and catches common mistakes; Expo config matches the stack vs generic presets alone. Source: https://github.com/eslint/eslint — Author: ESLint team — Docs: https://eslint.org/docs/latest/

UI framework & runtime

React 19.1.0 — Component model and hooks for screens and UI. Why: Industry standard for cross-platform UI; pairs with React Native. Source: https://github.com/facebook/react — Author: Meta — Docs: https://react.dev/

React Native 0.81.5 — Native iOS/Android UI primitives. Why: True native views vs WebView-only apps. Source: https://github.com/facebook/react-native — Author: Meta — Docs: https://reactnative.dev/docs/getting-started

Expo platform & modules (SDK ~54.0.33)

Expo (expo ~54.0.33) — Managed workflow, dev client, config (app.json), builds. Why: Faster iteration, unified config, and modules vs bare RN alone for a student/small team app. Source: https://github.com/expo/expo — Author: Expo — Docs: https://docs.expo.dev/

expo-router ~6.0.23 — File-based routing (app/ directory). Why: Matches web mental model and deep linking vs manual stack setup. Source: https://github.com/expo/expo/tree/main/packages/expo-router — Author: Expo — Docs: https://docs.expo.dev/router/introduction/

expo-image ~3.0.11 — Optimized image loading/caching. Why: Better perf than core Image for feeds and listing photos. Source: https://github.com/expo/expo/tree/main/packages/expo-image — Author: Expo — Docs: https://docs.expo.dev/versions/latest/sdk/image/

expo-image-picker ~17.0.10 — Camera and photo library for quick listing photos. Why: Cross-platform permissions and picker vs writing native modules. Source: https://github.com/expo/expo/tree/main/packages/expo-image-picker — Author: Expo — Docs: https://docs.expo.dev/versions/latest/sdk/imagepicker/

expo-linking ~8.0.11 — URL schemes and deep links (e.g. OAuth callbacks). Why: Works with Supabase OAuth redirect flow. Source: https://github.com/expo/expo/tree/main/packages/expo-linking — Author: Expo — Docs: https://docs.expo.dev/versions/latest/sdk/linking/

expo-auth-session ~7.0.10 — OAuth session helpers (often with expo-web-browser). Why: Structured OAuth handling vs hand-rolled flows. Source: https://github.com/expo/expo/tree/main/packages/expo-auth-session — Author: Expo — Docs: https://docs.expo.dev/versions/latest/sdk/auth-session/

Backend & data (Supabase)

@supabase/supabase-js ^2.100.0 — Client for Postgres, Auth, Storage, and Realtime. Why: One SDK for DB + buckets + auth vs stitching REST + S3 + custom auth. Source: https://github.com/supabase/supabase-js — Author: Supabase Inc. — Docs: https://supabase.com/docs/reference/javascript/introduction

Local persistence

@react-native-async-storage/async-storage 2.2.0 — Simple key-value storage (e.g. listing counts / flags). Why: Official community package vs deprecated **@react-native-community/async-storage** paths; not for highly sensitive tokens (Secure Store used for session). Source: https://github.com/react-native-async-storage/async-storage — Author: React Native Community — Docs: https://react-native-async-storage.github.io/async-storage/

A list of each data type that you will be storing in a database (i.e. long-term storage). Explain the purpose of each field in the database. How much storage (in bytes) will it require?

PostgreSQL: posts row (what insertPost actually writes for a new sale)

id is a UUID primary key generated by the database (16 bytes on disk). user_id is the seller's UUID, matching public.users.id (16 bytes); it ties the listing to whoever is logged in. type is the literal 'sale' (a few bytes of text plus Postgres varlena overhead). title holds the short listing name (tens to low hundreds of bytes depending on length). description is optional body copy; it can be null or empty, or grow to a few KB if the seller types more (variable text). price is a numeric (typically on the order of 8–16 bytes in Postgres, variable by precision). budget exists on the table for ISO posts but is not set by quick listing, so for this story it stays null and contributes negligible stored payload. condition and category are short constrained strings (on the order of 10–30 bytes each). images is a text[] of public URLs pointing at Supabase Storage objects—not the image bytes themselves; with a handful of URLs you usually budget roughly 1–2 KB total for the array, scaling with URL length and photo count. hashtags is another text[] of short tags (often tens to a few hundred bytes). created_at is a timestamptz (8 bytes). sold defaults to false (1 byte). boosted_until is null on a normal new listing unless you set boosts elsewhere (null uses almost no per-column storage in practice). Altogether, a typical quick-listing row is on the order of a few KB—mostly description plus the URL strings—not counting row header and alignment overhead (roughly tens of bytes extra per row).

Supabase Storage (long-term, but not the SQL database)

The binary photo files for quick listing are stored in the listing-images bucket. Their size is whatever you upload (often hundreds of KB to a few MB per image). Postgres only stores the string URLs in posts.images, not those bytes.

So, strictly for this user story: the database side is mainly one new posts row plus an existing users row referenced by user_id; image payloads are in object storage, with byte counts driven by file size, not by the URL fields in Postgres

A list of the user-visible and internally-visible effects if your frontend application:

Process crash

User-visible: App disappears or returns to the home screen; any unsent draft (photos, title, etc.) in memory is usually lost unless you had persisted it (you generally do not for the create form). After relaunch, user may need to sign in again if the session restore path is slow or fails.

Internal: OS terminates the JS/native process; no graceful teardown; Sentry/console won't get a final "cleanup" unless you had crash reporting. Pending in-flight fetch/Supabase calls are aborted mid-flight.

Lost all runtime state

User-visible: Same session as "cold start": feed may reload empty then refill; form fields reset; navigation stack resets to initial route; spinners while data refetches. If auth session is only in memory in some edge case, user could look "logged out" until SecureStore is read again.

Internal: All useState / context cleared; subscriptions (e.g. Realtime channels) dropped until remounted; no in-memory cache unless rebuilt from network or disk.

Erased all stored data (app data cleared / uninstall / wipe AsyncStorage + SecureStore)

User-visible: User is logged out (session gone from SecureStore); local listing limits / Pro flags from AsyncStorage reset; no offline cache; everything refetched from server if they sign in again.

Internal: All local keys gone; next launch runs full auth bootstrap; analytics identity may reset unless tied to server account after re-login.

Some data in the database appears corrupt

User-visible: Broken images (bad URLs), wrong prices, garbled text, missing seller on a card, crashes or error toasts if parsing assumes valid shapes; list screens may show empty slots or "Something went wrong."

Internal: mapRowToPost or similar may throw or produce partial objects; Supabase client returns 200 with bad payload; logs show JSON shape errors; you may need admin SQL or backups to fix source of truth.

Remote procedure call failed (Supabase RPC or rpc() call)

User-visible: Feature that depends on that RPC fails: error message, retry button, or silent failure with stale UI.

Internal: HTTP error / Postgres error surfaced in error object; client may retry or not depending on code; server logs show the failed function and stack.

Client overloaded (CPU pegged, main thread busy, too much JS work)

User-visible: Janky scrolling, frozen taps, keyboard lag, “app not responding” on Android; photos picker may feel slow.

Internal: Frame drops in perf monitors; long tasks blocking the JS thread; no server symptom unless the client spams requests.

Client out of RAM

User-visible: iOS/Android may kill the app in the background first; foreground may crash or reload; large image carousels or many decoded bitmaps worsen this.

Internal: OOM in native logs; process death; no DB corruption from this alone.

Database out of space

User-visible: New listings, messages, or signups fail; user sees generic errors (“try again”) or specific message if you map it; reads might still work briefly.

Internal: Postgres INSERT/UPDATE errors; Supabase returns 5xx or constraint/storage errors; replication and backups may also fail ops-side.

Lost network connectivity

User-visible: Spinners time out; “offline” or error alerts; feed empty or stale; cannot create listing (upload + insert need network); cached images from expo-image may still show if on disk.

Internal: fetch failures, Supabase client errors; Realtime disconnects; queued retries depend on whether you implemented them (often none for writes).

Lost access to its database (wrong keys, project paused, IP ban, outage)

User-visible: Same as bad network for most users: nothing loads, auth may fail, all mutations error.

Internal: 401/403/5xx from Supabase; client logs show auth or connection errors; Storage calls fail too if same project.

Bot signs up and spams users

User-visible: Legitimate users get notification/message noise, junk listings in feed, or degraded trust; possible rate-limit errors if you add throttling.

A list all Personally Identifying Information

Item	Why keep it	How stored	How it entered	Path into storage	Path out of storage
Email	Account identity, password/OAuth recovery, uniqueness in auth	Supabase Auth (auth.users); not in your public.users unless you duplicate elsewhere	Email/password sign-up or Google OAuth (create-profile / lib/supabase.ts: authenticate, signInWithOAuth, exchangeCodeForSession)	authenticate / OAuth → Supabase Auth → auth.users	Session used by @supabase/supabase-js (supabase/client.ts); profile UI reads user from AuthContext
Stable user id (UUID)	Tie listings, profiles, messages to one person	UUID in auth.users.id and public.users.id (FK)	Same auth flows; saveProfile upserts public.users with id from auth	saveProfile in lib/supabase.ts (dbPayload.id)	fetchFeedPosts, insertPost, messaging queries in supabase/queries.ts join/filter on user_id
Display name	Shown on profile and listings	public.users.display_name (text)	create-profile.tsx form → saveProfile	Form state → saveProfile → Postgres	Feed mapping in mapRowToPost → Seller.displayName
Username	Public handle, uniqueness	public.users.username (text, unique)	create-profile → saveProfile	Same	Feed → Seller.username
Location / zip	“Verified” local trust (your copy calls out zip)	public.users.location (text)	create-profile zip field → saveProfile	Same	Feed Seller.location; discovery if used
Avatar URL	Profile/feed image	public.users.avatar_url (text); may be Google URL or local URI string until you	create-profile (Google metadata or ImagePicker → setAvatarUrl) → saveProfile	Same	Feed / profile Image / expo-image loads URI

		upload to Storage			
Listing content + user_id	Attribution	posts: title, description, optional contact-like text; user_id links to seller	create-listing → submitListing → insertPost	Shown fields + useAuth().user?.id	fetchFeedPosts → SalePostCard etc.

Who secures each store: your repo doesn't define owners, so assign named people now. At minimum: a Supabase owner (Auth/DB/Storage settings and key rotation), a backend/infra owner (RLS, backups, envs), a mobile owner (no secrets in client, SecureStore usage), and a compliance owner (retention/privacy process). Replace those roles with actual teammate names.

Auditing access: keep routine access tight (named maintainers only, MFA required, no service-role key in the client, periodic access reviews for Supabase and GitHub). For non-routine production access, require a ticket + approval, time-box credentials, and log who accessed what and why. Prefer read replicas/controlled exports over ad-hoc production SQL. Also, your current public tables were observed with RLS off—treat that as a production blocker.

Minors: there is no explicit age gate in the reviewed flows, so minors could still sign up via email/OAuth. Guardian consent is not implemented. If minors are in scope, you need a separate parental-consent product/legal flow. Screening or restricting access for people convicted/suspected of child abuse is a legal/operations policy area, not enforced by current app code without an external trust-and-safety process/tooling.